Synthetic Homotopy Theory

Abbreviated lecture notes

Midlands Graduate School (MGS)
13–17 April 2026, Nottingham, UK

by
Axel Ljungström and Aref Mohammadzadeh
School of Computer Science
University of Nottingham

Heavily based on notes from MGS 2024 by Ulrik Buchholtz and Mark Williams

Lecture 1

What is homotopy theory?

Homotopy theory is the study of paths between points (and paths between these paths, and so on) in topological spaces.

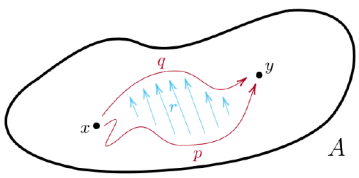
What is synthetic homotopy theory?

Synthetic homotopy theory is the study of the identity type in Martin-Löf Type Theory.

What is the connection?

Awodey, Hoffmann, Streicher, Voevodsky, Warren[1]:

- Turns out: types in type theory can be interpreted as spaces
  - 'A is a type' (or $A : \mathcal{U}$) —> 'A is a topological space (up to *homotopy*, i.e. continuous deformation, but we don't need to worry about this now)'
  - $x : A$ —> 'x is a point in A'.
  - $p : x =_A y$ —> 'p is a (continuous) path from x to y in A'.
  - $r : p =_{x=y} q$ —> 'p is a *higher* path/continuous deformation/*homotopy* from p to q'



While all paths may be interesting to homotopy theorists, there are certainly quite a lot of them... One thing that homotopy theorists like to do is to restrict

[1] Voevodsky is the person who is directly attributed this connection, but the other authors have also made crucial contributions that inspired this interpretation

to a certain classs of paths, namely those that are *loops*, i.e. start and end on the same point.

Draw pic

Given a space A and $x \in A$, we can define

$$\pi_1(A, x) := \{\text{Loops in } A \text{ starting and ending at x}\}/\sim$$

where we quotient out by *homotopy* – two loops are equal if one can be continuously deformed into the other. The reason that we restrict to loops is that this gives us a group structure. Indeed, given two loops p and q, we can compose these ('walk along p and then along q'). Similarly, we can revert them: p^{-1} = *'p, but backwards'*. We also get a neutral element const_x which is the constant loop standing still on x.

The group $\pi_1(A, x)$ is called the *fundamental group* of A (based at x). Now, why stop at 1-dimensional loops from x to itself? We can easily (although we won't do it here) define the notion of an n-dimensional loop and can similarly define

$$\pi_n(A, x) = \{n\text{-dimensional loops in } A \text{ touching } x\}/\sim$$

where we again quotient by an appropriate notion of homotopy. This is called the *nth homotopy group of A (based at x)*. Knowing $\pi_n(A, x)$ a space A and suitable $x \in A$ for all n esentially tells you 'all you need to know' about your space (if you're a homotopy theorist, that is). Unfortunately, these are *incredibly hard* to compute in general. For this reason, homotopy theorists often focus on the special case when A is the m-dimensional (hyper-)sphere S^m. In a way, all spaces can ('up to homotopy') be thought of as consisting of a bunch of spheres of varying dimension glued together, this seems like a very suitable thing to study. Unfortunately, also computing $\pi_n(S^m, \star)$ for arbitrary $n, m \geq 1$ is notoriously hard – or perhaps this is fortunate, since computing these has, in a way, become the raison d'etre of homotopy theory.

The goal of this course will be to explore whether we can compute any of these $\pi_n(S^m, \star)$ in synthetic homotopy theory, i.e. using the language of dependent type theory or, actually, *Homotopy Type Theory* (HoTT), an extension of this language). In thislecture, we will define homotopy groups in HoTT. In lecture 2, we introduce some actual spaces into our type theory (in particular spheres). In lectures 3 and 4 we will compute some homotopy groups of these spaces – in particular of the circle and the 3-sphere.

Note, that HoTT is just dependent type theory with *univalence* and *higher inductive types*. I'll introduce these concepts as we go.

Paths

Let us recall what is probably the most important notion from dependent type theory:

Def (Path induction)**.** Let $x : A$. In order to prove a statement/construct an element of type $(y : A)(p : x = y) \to B(y, p)$, it suffices to prove/construct an element of $B(x, \mathrm{refl}_x)$.

Let's give some examples of this principle in action:

Ex (Transport)**.** Given $p : x =_A y$ and $B : A \to \mathcal{U}$, we get a map $\mathrm{transport}^B_p : B(x) \to B(y)$ by path induction on p:

$$\mathrm{transport}^B_{\mathrm{refl}_x} : B(x) \to B(x) \quad := \quad \mathsf{id}_{B(x)}$$

draw picture explaining transport

Ex (Function application)**.** Given $f : X \to Y$ and $p : x = y$, we have $\mathrm{ap}_f(p) : f(x) = f(y)$ (by path induction on p).

Question. Function application is only stated for plain functions, but we're in dependent type theory. What if $f : (x : X) \to B(x)$ is a dependent function?

We can't write $\mathrm{ap}_f(p) : f(x) = f(y)$ because $f(x) : B(x)$ and $f(y) : B(y)$, so this equality doesn't type check! But $B(x)$ and $B(y)$ are equal types (by p), so we should still be able to express the idea that $f(x)$ and $f(y)$ are 'equal' (modulo p):

Def (Dependent paths)**.** Given $B : X \to \mathcal{U}$ dep. type, $p : x_0 = x_1$, and terms $b_0 : B(x_0)$ and $b : B(x_1)$, define

$$(b_0 =^B_p b_1) := (\mathrm{transport}^B_p(b_0) = b_1)$$

Note that the above definition coincides *exactly* with the usual identity type whenever p is refl.

Ex (Function application)**.** Given $f : X \to Y$ and $p : x = y$, we have $\mathrm{apd}_f : (p : x = y) \to f(x) =^B_p f(y)$ (by path induction on p).

Remember that homotopy theorists were interested in composing paths and inverting paths? In HoTT, this idea simply corresponds to the fact that our identity type is transitive and symmetric.

Ex (Path composition/inversion)**.** Given $p : x = y$ and $q : y = z$, we can define $p \cdot q : x = z$ and $p^{-1} : y = z$ by path induction on p. These operations satisfy (for suitable p, q and r):

- $p \cdot p^{-1} = \mathrm{refl}$
- $p^{-1} \cdot p = \mathrm{refl}$
- $p \cdot \mathrm{refl} = p$
- $\mathrm{refl} \cdot p = p$
- $(p \cdot q) \cdot r = p \cdot (q \cdot r)$

Are we there yet?

So, we have path types with notions of composition, inversion, and a neutral element. Why not just play the same game as in traditional homotopy theory and set $\pi_1(X, x) := (x = x)$ with group operations given by path composition and path reversal. This is *almost* right. The issue: the group laws hold but they can hold in *infinitely many ways*! Suppose you and I both are tasked with defining $\pi_1(X, x)$ in type theory. Both of us provide the same underlying type, the same operations, and the same neutral element – but we have provided different proofs of the fact that, say, $p \cdot p^{-1} = \text{refl}$. Which definition is the appropriate one? Mine or yours? To arrive at a well-defined notion of groups, we would like to make sure that this situation never arises – we would like to 'quotient' by all 'higher paths' (i.e. paths involving other paths, such as the path witnessing $p \cdot p^{-1} = \text{refl}$. The next part of this lecture is about how to construct these kinds of quotients.

n-types

In order to even state that we wish to 'quotient out higher paths', we need a way to pinpoint what we even mean by 'higher paths'. A good way of capturing a type having 'higher structure' in general is the notion of an n-type. Let us start with two special cases:

Def. A type X is **contractible** if the type

$$\text{isContr}(X) := (x : X) \times ((y : X) \to x =_X y)$$

is inhabited.

Def. A type X is a **proposition** if the type

$$\text{isProp}(X) := (x, y : X) \to x =_X y$$

is inhabited.

Usin this, we can define what it means to be an n-type:

Def. A type X is

- a **(-2)-type** if isContr A,
- a **(-1)-type** if isProp A, and
- an **n-type** if $(x, y : X) \to \text{is-}(n-1)\text{-type}\ (x =_A y)$.

We may use the following conventions

Convention.

- 0-types are called **Sets**

- 1-types are called **Groupoids**

To get some intuition (hopefully, you will get even more later on in the course), let us give some 'concrete' examples.

Ex.

- All (-2)-types:
  - The unit type 1. Actually, all (-2)-types are equivalent to this type.
  - The **singleton** type (over some $x : X$) i.e. $(y : X) \times (x = y)$ is contractible.
- Propositions ((-1)-types):
  - $1, \emptyset$ (and everything in between)
  - The statement that a type is an n-type is itself a (-1)-type: $\mathsf{isProp}(\text{is-}n\text{-type}\ (X))$.
  - The **singleton** type (over some $x : X$) i.e. $(y : X) \times (x = y)$ is contractible.
  - We haven't come this far yet, but we will soon define what it means for a function to the an equivalence (roughly, 'invertible'). The statement that $f : A \to B$ is an equivalence is a (-1)-type.
- Sets (0-types). These are types that are discrete in nature – they can have a bunch of points but no interesting paths (i.e. no interesting homotopical structure):
  - Any (plain) inductive type, e.g. $\mathbb{Z}$
  - cf. Hedberg's theorem if you're interested in how to prove this
- n-type:
  - The universe of n-types, $\mathcal{U}^{\leq n} := (X : \mathcal{U}) \times \text{is-}n\text{-type}\ (X)$, is an $(n+1)$-type.
  - Any n-type is also an $(n+1)$-type.

We said before that we'd like to be able to 'quotient' by all 'higher structure' of a type. In other words, we would like to be able to take a type and turn it into an n-type. This is precisely the role of the n-truncation. Here, it is stated as a definition, although we never show how it's defined... For now, take it for granted:

Def. Given a type X, there is an n-type $\|X\|_n$ equipped with a map $|-|_n : X \to \|X\|_n$ s.t. for every $f : X \to Y$ where Y is an n-type, we have

$$\begin{tikzcd} X \arrow[r, "f"] \arrow[d, "|-|_n"'] & Y \\ \|X\|_n \arrow[ur, dashed] & \end{tikzcd}$$

Alternatively:

$$_ \circ |_|_n : (\|X\|_n \to Y) \to (X \to Y)$$

is an equivalence. This phrasing might be better because we easily are able to generalise it to get the induction principle (i.e. dependent elimination) for any dependent (n-)type $B : X \to \mathcal{U}^{\leq n}$: we require that

$$_ \circ |_|_n : ((x : \|X\|_n) \to B(x)) \to ((x : X) \to B(|x|_n))$$

is an equivalence.

- Common notation: $\|X\| := \|X\|_{-1}$

Finally: Homotopy groups

Def. A **pointed type** is a pair $(X, \star_X) : \underbrace{(X : \mathcal{U}) \times X}_{\mathcal{U}_\star}$ where $X : \mathcal{U}$ and $\star_X : X$.

Def. A **pointed map** is a pair $(f; \star_X) : \underbrace{(f : X \to Y)(\times f(\star_X) = \star_Y)}_{(X,\star_X) \to_\star (Y,\star_Y)}$

Notation.

- Often simply write $X : \mathcal{U}_\star$ and $f : X \to_\star Y$, i.e. we leave basepoints and pointedness proofs implicit
- Always write $\star_X : X$ for basepoints and $\star_f : f(\star_X) = \star_Y$ for pointedness proofs.

Def. We define the **loop space** of a pointed types X by

$$\Omega(X) := (\star_X = \star_X)$$

This type is itself pointed by $\mathrm{refl}_{\star_X}$.

- It is **functorial**: given $f : X \to_\star Y$, we define $\Omega(f) : \Omega(X) \to \Omega(Y)$ by

$$\Omega(f)(p) := (\star_f)^{-1} \cdot \mathrm{ap}_f(p) \cdot \star_f$$

- Path composition defines a 'group structure' on Ω
  - Problem: group laws only hold up to homotopy.
    - For instance there might be more than one element of $p \cdot p^{-1} = \mathrm{refl}_{\star_X}$.
  - Solution: need to truncate

Def. The **first homotopy group** (also the **the fundamental group**) of a pointed type X is defined by $\pi_1(X) := \|X\|_0$

- Multiplication is inherited immediately from path composition on $\Omega(X)$. By the univeral property: $|p|_0 \cdot |q|_0 = |p \cdot q|_0$

Note $\Omega : \mathcal{U}_\star \to \mathcal{U}_\star$ – can be iterated.

Def. The nth homotopy group of $X : \mathcal{U}_\star$ is defined by

$$\pi_n X := \|\Omega^n X\|_0$$

Note that $n \geq 1$!

- Ω is functorial $\implies$ π_n is functorial
- Write $f^* := \pi_n(f) : \pi_n(X) \to \pi_n(Y)$ for its functorial action.

Things I should've talked about but didn't

I didn't have time to mention that there is a notion of n-connected types. This notion is, in a sense, dual to the notion of n-type:

Def. A type X is **n-connected** if $\|X\|_n$ is contractible.

We often simply write 'connected' instead of '0-connected'.

Proposition. *If X is $(n \geq -1)$-connected, then $x =_X y$ is $(n-1)$-connected for all $x, y : X$.*

Just like types, functions can also be 'n-types' (we call these n-truncated) and 'n-connected'. This essentially means that they behave nicely when mapping into/out out of n-types/n-connected types. To define this notion, we need fibres:

Def. The **fibre** of a function $f : X \to Y$ over a point $y : Y$ is the type

$$\mathrm{fib}_f(y) := (x : X) \times (f(x) = y)$$

Def. A map $f : X \to Y$ is:

1. **n-truncated** if $\mathrm{fib}_f(y)$ is an n-type for all $y : Y$,
2. **n-connected** if $\mathrm{fib}_f(y)$ is n-connected for all $y : Y$.

These notions are crucial, but their significance is perhaps most easily illustrated via the following results (our very first homotopy group computations!):

Proposition. *If X is an n-type, then $\pi_k(X)$ vanishes $\forall k > n$.*

Corollary. *$\pi_{n\geq 1}(X)$ vanishes for any set X. In particular for $X = 1$.*

Proposition. *If X is n-connected, then $\pi_k(X)$ vanishes $\forall k \leq n$.*

Recap: univalence

I never stated the univalence axiom, and that would probably have been appropriate for a course basde on homotopy type theory. To state it, we need to define equivalences:

Def. A function $f : X \to Y$ is an **equivalence** if $\mathrm{fib}_f(y)$ is contractible for all y.

- Let $(X \simeq Y) \times \mathrm{isEquiv}(f)$
- Fact: $\mathrm{id}_X : X \to X$ is an equivalence.

Remark. Why not use $\underbrace{(g : (Y \to X)) \times (g \circ f = \mathrm{id}) \times (f \circ g = \mathrm{id})}_{:=\mathrm{qinv}(f)}$ as definition? The issue with this definition is that it is *not a proposition*, so there may be many different proofs of it for a given function f. This makes it inappropriate to use. However, it turns out that $\mathrm{qinv}(f)$ and $\mathrm{isEquiv}(f)$ both imply each other (i.e. they are *logically equivalent*), so we can just pretend that these definitions are the same. That's certainly what I do when I construct (most) equivalences. For this reason, I will often talk about the 'inverse' of an equivalence.

Axiom (Univalence)**.** *The canonical map $X =_{\mathcal{U}} Y \to (X \simeq Y)$ is an equivalence. In particular, $X \simeq Y \to X = Y$*

Above, the 'canonical map' is defined using an appropriate transport or by path induction. Constructing it is a good exercise if you feel uncertain about these concepts.

Fact. Univalence implies function extensionality, i.e. the statement that if $(((xy : X) \to f(x) = g(x))$ then $f = g$

From now on, we will always assume (and use) univalence/function nextensionality, often without comment.

Lecture 2: A menagerie of higher inductive types

One of the key goals of this course is to compute homotopy groups $\pi_n(A)$. Now, we know how to define this group, but it'd also be great if we could stick in some interesting candidate for what A could be. In particular, we're interested in setting $A := S^n$, i.e. the n-dimensional sphere. These are defined as *higher inductive types* (HITs) – a new addition to type theory which, together with univalence, is what *makes* HoTT. These are inductive types where we are also allowed to introduce *path* constructors. These are best introduced through examples, as writing down a fully general speicification of what a HIT should be is rather involved. As we are interested in spheres, why not start with the cirlce?

The circle

Def. The **circle**, S^1, is the HIT generated by

- A point $\star_{S^1} : S^1$
- A path loop : $\star_{S^1} = \star_{S^1}$

Draw it

Mapping out of S^1 is just like mapping out of a usual inductive type – it comes with recursion/induction rules.

Recursion: A map $f : S^1 \to X$ is defined by the following data:

- $x : X$
- $p : x =_X x$

The rule says that f defined this way satisfies $f(\star_{S^1}) := x$ and $\mathrm{ap}_f(\mathrm{loop}) = p$.

Ex. We can define the map $\mathsf{double} : S^1 \to S^1$ by

$$f(\star_{S^1}) := \star_{S^1}$$
$$\mathrm{ap}_f(\mathrm{loop}) := \mathrm{loop} \cdot \mathrm{loop}$$

A nice way of stating the recursion principle is to say that the evaluation map

$$\begin{aligned}(S^1 \to X) &\to (x : X) \times (x = x)\\ f &\mapsto (f(\star_{S^1}), \mathrm{ap}_f(\mathrm{loop}))\end{aligned}$$

is an equivalence. This is nice, because it's immediately generalisable to the dependent case:

Induction: For $B : S^1 \to \mathcal{U}$, The evaluation map

$$\begin{aligned}((x : S^1) \to B(x)) &\to (x : X) \times (x =^B_{\mathrm{loop}} x)\\ f &\mapsto (f(\star_{S^1}), \mathrm{apd}_f(\mathrm{loop}))\end{aligned}$$

is an equivalence.

Pushouts

The circle is a great space, but it's probably best to introduce some other ones too, to make life a bit more interesting for ourselves. I said before that I didn't want to define a notion of arbitrary HIT, so we can't introduce 'all' spaces. However, we can still define a very large class of them. These are built up using *pushouts*:

Def. Given a span $B \xleftarrow{f} A \xrightarrow{g} C$ we can form its **pushout** by the higher inductive type $B \sqcup^A C$ with the following constructors:

$$\begin{aligned}\mathrm{inl} &: B \to B \sqcup^A C\\ \mathrm{inr} &: C \to B \sqcup^A C\\ \mathrm{glue} &: (a : A) \to \mathrm{inl}(f(a)) = \mathrm{inr}(g(a))\end{aligned}$$

Draw pictures

We write

$$\begin{tikzcd} A \arrow[r, "g"] \arrow[d, "f"'] \arrow[dr, phantom, "\ulcorner"] & C \arrow[d] \\ B \arrow[r] & D \end{tikzcd}$$

to express that D is the pushout $B \sqcup^A C$ defined above. Let's state the recursion principle *informally* (I'll leave it as an exercise to deduce the induction principle, i.e. the dependent version of the recursion principle). The recursion principle for pushouts simply says that *in order to define a function $h : B \sqcup^A C \to X$, where the pushout is the one defined above, it suffices to construct two functions $l : B \to X$ and $r : C \to X$, and provide a proof that for any $a : A$, we have that $l(f(a)) = r(g(a))$.* Let's give some examples of pushouts:

Def (Suspensions)**.** ΣX, the **suspension** of X is defined as the pushout:

$$\begin{tikzcd} X \arrow[r] \arrow[d] & 1 \arrow[d] \\ 1 \arrow[r] \arrow[ur, phantom, "\ulcorner"] & \bullet \end{tikzcd}$$

This construction is so important (in particular, we will need it to define spheres), that we give its constructors special names.

- $N := \mathrm{inl}(\star_1)$ – the 'north pole'
- $S := \mathrm{inr}(\star_1)$ – the 'south pole',
- $\mathrm{merid}(x) := \mathrm{glue}(x)$ – 'meridians from north to south (through each point $x : X$)'
- We always take ΣX to be pointed by N.

draw a picture of ΣS^1

Def. We define the n-sphere inductively by:

$$S^{-1} = \emptyset \qquad S^{n+1} = \Sigma S^n$$

It's really nice that spheres are suspensions, because suspensions are (usually) really nice:

Proposition (Loop Space–Suspension Adjunction)**.** *Let X and Y be a pointed type. We have*

$$(\Sigma X \to_\star Y) \simeq (X \to_\star \Omega(Y))$$

Proof. Unfold universal property of ΣX:

$$\begin{aligned}
(\Sigma X \to_\star Y) &\equiv (f : \Sigma X \to Y) \times (f(N) = \star_Y) \\
&\simeq (y_N : Y) \times (y_S : Y) \times (X \to y_N = y_S) \times (y_N = \star_Y) \\
&\simeq \underbrace{(y_N : Y) \times (y_N = \star_Y)}_{\text{contr. at } (\star_Y, \mathrm{refl})} \times (y_S : Y) \times (X \to y_N = y_S) \\
&\simeq (y_S : Y) \times (X \to \star_Y = y_S) \\
&\simeq (y_S : Y) \times (p : X \to \star_Y = y_S) \times (q : \star_Y = y_S) \times (q = p(\star_X)) \\
&\simeq \underbrace{(y_S : Y) \times (q : \star_Y = y_S)}_{\text{contr. at } (\star_Y, \mathrm{refl})} \times (p : X \to \star_Y = y_S) \times (q = p(\star_X)) \\
&\simeq (p : X \to \star_Y = \star_Y) \times (\mathrm{refl} = p(\star_X)) \\
&\simeq (X \to_\star \Omega Y)
\end{aligned}$$

□

Another nice property is that suspensions increase connectedness:

Theorem (Connectivity of suspension)**.** *If X is n-connected then ΣX is $n+1$-connected.*

Proof. Exercise (use the exercise on connectedness vs diagonal maps). □

This allows us to compute our very first homotopy groups of spheres!

Corollary. *The sphere S^n is $(n-1)$-connected.*

Proof.
- S^{-1} is (-2)-connected (all types are).
- $S^{n+1} = \Sigma^{n+1}\emptyset$, so by previous lemma this is $((-2)+(n+1)) = (n-1)$-connected. □

Corollary. $\pi_m(S^n) = 1$ *when* $m \leq n-1$.

Other pushouts There are many other interesting examples of pushouts. We will, in particular, use the *join*:

Joins

$$\begin{tikzcd} X \times Y \arrow[r] \arrow[d] & Y \arrow[d] \\ X \arrow[r] \arrow[ur, phantom, "\ulcorner"] & X * Y \end{tikzcd}$$

Wedge sums (of pointed types)

$$\begin{tikzcd} X \arrow[r] \arrow[d] & Y \arrow[d] \\ X \arrow[r] \arrow[ur, phantom, "\ulcorner"] & X \vee Y \end{tikzcd}$$

The cofibre of a function $f : X \to Y$

$$\begin{tikzcd} X \arrow[r] \arrow[d, "f"'] & 1 \arrow[d] \\ Y \arrow[r] \arrow[ur, phantom, "\ulcorner"] & \mathsf{cofib}_f \end{tikzcd}$$

The smash product (of pointed types)

$$\begin{tikzcd} X \vee Y \arrow[r] \arrow[d, "\iota"'] & 1 \arrow[d] \\ X \times Y \arrow[r] \arrow[ur, phantom, "\ulcorner"] & X \wedge Y \end{tikzcd}$$

The three-by-three lemma and applications

Most (OK, many) of the spaces we are interested in are pushout. Is there a nice and flexible way of proving that two pushouts are equivalent? Yes! The tool we'll use for this is called the 3×3-lemma; if I could go back in time, I'd probably convince Guillaume Brunerie and Dan Licata (who first introduced it in HoTT – it was, of course, already known by classical homotopy theorist) to call it the 3×3-theorem. To make sense of the statement, we'll need to state the functoriality of pushouts: given a commutative diagram

$$\begin{tikzcd}
B \arrow[d, "f"'] & A \arrow[l] \arrow[r] \arrow[d, dashed, "h"] & C \arrow[d, "g"] \\
Y & X \arrow[l] \arrow[r] & Z
\end{tikzcd}$$

we get an induced map $f \sqcup^h g : B \sqcup^A C \to Y \sqcup^X Z$. It's a good exercise to construct it (it follows easily from the recursion principle for pushouts).

Let's state the 3×3-lemma. In what follows, squiggly arrows indicate 'taking pushouts'. Consider a commutative diagram of the following form:

$$\begin{tikzcd}
A & B \arrow[l, "f"'] \arrow[r, "g"] & C \\
D \arrow[u, "\alpha"] \arrow[d, "\delta"'] & E \arrow[l, "h"'] \arrow[r, "i"] \arrow[u, dashed, "\beta"] \arrow[d, dashed, "\epsilon"] & K \arrow[u, "\gamma"'] \arrow[d, "\eta"] \\
L & M \arrow[l, "j"] \arrow[r, "k"'] & N
\end{tikzcd}$$

We can take pushouts either of the vertical spans or the diagonal spans. This will give us two new spans (of pushouts and maps between pushouts) which we can again take the pushout of:

$$\begin{tikzcd}
 & & & P_2 \\
A \arrow[d, squiggly] & B \arrow[l, "f"'] \arrow[r, "g"] \arrow[d, squiggly] & C \arrow[r, squiggly] \arrow[d, squiggly] & A \sqcup^B C \arrow[u, squiggly] \\
L \sqcup^D A & B \sqcup^E M \arrow[l, "f \sqcup^h j"] \arrow[r, "g \sqcup^i k"'] & C \sqcup^K N \arrow[r, squiggly] & P_1
\end{tikzcd}$$

$$\begin{tikzcd}
A & B \arrow[l, "f"'] \arrow[r, "g"] & C \arrow[r, squiggly] & A \sqcup^B C \\
D \arrow[u, "\alpha"] \arrow[d, "\delta"'] & E \arrow[l, "h"'] \arrow[r, "i"] \arrow[u, dashed, "\beta"] \arrow[d, dashed, "\epsilon"] & K \arrow[u, "\gamma"'] \arrow[d, "\eta"] \arrow[r, squiggly] & D \sqcup^E K \arrow[u, "\alpha \sqcup^\beta \gamma"'] \arrow[d, "\delta \sqcup^\epsilon \eta"] \\
L & M \arrow[l, "j"] \arrow[r, "k"'] & N \arrow[r, squiggly] & L \sqcup^M N
\end{tikzcd}$$

Lemma (3×3-lemma)**.** $P_1 \simeq P_2$

In fact, this equivalence is natural but I don't think anyone has ever bothered writing it down. We won't prove the 3×3-lemma here. It is completely direct, but technical – the kind of thing you do once and never again...

Let's consider an application. For it, we need a quick lemma:

Lemma. *Given an equivance $e : A \simeq B$, the following is a pushout square:*

$$\begin{tikzcd} A \arrow[r, "g"] \arrow[d, "e"'] & C \arrow[d] \\ B \arrow[r] \arrow[ur, phantom, "\ulcorner"] & C \end{tikzcd}$$

Proof. By univalence, we have that $(A \simeq B) \simeq (A = B)$, so we may assume that $e = \mathsf{univalence}(p)$ for some $p : A = B$. Applying path induction on p, we may assume that e is of type $A \simeq A$ and is given by univalence applied to $\mathrm{refl}_A : A = A$. It follows that $e = \mathrm{id}_A$ (do you see why? – it follows from the somewhat convoluted statement of the univalence axiom). Hence, we only need to prove that

$$\begin{tikzcd} A \arrow[r, "g"] \arrow[d, "\mathsf{id}_A"'] & C \arrow[d] \\ A \arrow[r] & C \end{tikzcd}$$

is a pushout. The recursion principle for pushouts gives us:

$$\begin{aligned} (A \sqcup^A C \to C) &\simeq (r : C \to C) \times (l : A \to C) \times (r \circ g = f \circ \mathrm{id}_A) \\ &= (r : C \to C) \times \underbrace{(l : A \to C) \times (r \circ g = f)}_{\text{contractible}} \\ &= (C \to C) \end{aligned}$$

So we can provide $\mathrm{id}_C : C \to C$ to get a map $h : A \sqcup^A C \to C$. One can easily check that, because $\mathrm{id}_C : C \to C$ is an equivalence, so is $h : A \sqcup^A C \to C$. $\square$

Note that we actually don't need univalence for the above proof; working without it, however, makes the proof a bit trickier.

As our first application of the 3×3-lemma, we get another crucial principle from classical homotopy theory: the pushout-pasting lemma:

Proposition (Pushout-pasting)**.** *Consider a commutative diagram of the form:*

$$\begin{tikzcd} A \arrow[r] \arrow[d] & C \arrow[r] \arrow[d] & E \arrow[d] \\ B \arrow[r] \arrow[ur, phantom, "\ulcorner"] & D \arrow[r] & F \end{tikzcd}$$

TFAE:

- *F is a pushout of the rightmost square*
- *F is a pushout of the outermost square*

Proof. Consider the following instance of the 3×3-lemma

$$\begin{tikzcd}
 & & & F_1 \\
B & \emptyset \arrow[l] \arrow[r] & \emptyset \arrow[r, squiggly] & B \arrow[u, squiggly] \\
A \arrow[u] \arrow[d] & \emptyset \arrow[l] \arrow[r] \arrow[u] \arrow[d] & \emptyset \arrow[r, squiggly] \arrow[u] \arrow[d] & A \arrow[u] \arrow[d] \\
C \arrow[d, squiggly] & C \arrow[l, "\mathsf{id}_C"] \arrow[r] \arrow[d, squiggly] & E \arrow[r, squiggly] \arrow[d, squiggly] & E \\
F_2 & D \arrow[l, squiggly] & C \arrow[l] \arrow[r] & E
\end{tikzcd}$$

We get that F_1 (the pushout of the other square) is equivalent to F_2 (the pushout of the rightmost square). Hence, if one is a pushout, then so is the other. Note that we've applied the previous lemma several times when computing the pushouts of the rows/columns. □

Another very important application of the 3×3-lemma is the following:

Proposition. *For any types A, B, C, there is a natural equivalence*

$$(A * B) * C \simeq A * (B * C).$$

To prove this, we will need to know that the pushouts commute with products, i.e. if

$$\begin{tikzcd}
A \arrow[r, "g"] \arrow[d, "f"'] & C \arrow[d] \\
B \arrow[r] & D
\end{tikzcd}$$

is a pushout square, then so is

$$\begin{tikzcd}
A \times X \arrow[r, "g \times \mathsf{id}_X"] \arrow[d, "f \times \mathsf{id}_X"'] & C \times X \arrow[d] \\
B \times X \arrow[r] & D \times X
\end{tikzcd}$$

You can add this fact to the long list of 'exercises left for the reader'.